Dossier professionnel

MORAGNY Alban

Développement Fullstack

Certification Niveau 6 « Développeur en Intelligence Artificielle et Data Science » - RNCP 36581

Promotion Bachelor 3 Dev IA C1

10 Février 2025 - 25 Juillet 2025

Tuteur de stage : ACHBANI Mohamed

Entreprise : Montpellier Méditerranée Métropole - Hôtel de ville de Montpellier 34000

Sommaire

Sommaire 1
Introduction 4
I. Environnement professionnel 4
1. Présentation de la métropole 4
Historique et but de la Métropole de Montpellier 4
Chiffres clés 4
Organigramme du PND et du SSIM 5
2. Positionnement du stage 6
L'Unité Projet & Développement 6
Contexte de mon équipe 7
Transversalité du stage 8
Mon rôle au sein de l'équipe 8
3. Contexte du projet 9
Équipe du projet 9
Limites des fichiers Excel précédemment utilisés 9
Histoire du projet 10
Objectifs métiers et enjeux liés à la donnée 10
4. Objectifs du projet 11
5. Cartographie du système d'information 11
1. Architecture technique 11
2. Schéma réseau simplifié (déploiement en interne) 12
3. Utilisateurs et rôles 12
4. Données manipulées (tables principales) 12
5. Schéma relationnel simplifié 13
6. Technologies et méthodologies employées 14
1. Technologies principales 14
2. GitLab CI/CD 15
II. Valorisation des compétences 15
Mission 1 – Analyse des besoins et maquettage 15
Contexte initial 15
Recueil des besoins 16
Mission 2 – Authentification & Sécurité 18
Introduction 18
1. Authentification via api3m_url 19
2. Séparation frontend / backend 20
3. Gestion des rôles et autorisations 20

4. Cycle de vie des tokens & Interceptors Axios---------- 21
5. Tests et validations---------- 22
6. Retour critique---------- 24
Mission 3 – Interface utilisateur---------- 24
3.1 Structure des routes et pages----------25
3.2 Pages principales---------- 25
3.3 Composants SoftUI et responsive design----------29
3.4 Intégration des appels API----------30
3.5 Retours utilisateurs et pistes d'amélioration----------30
Mission 4 – Traitement des données---------- 31
4.1 Modélisation & création des tables---------- 31
4.2 Contraintes & validations----------32
4.3 Gestion des doublons---------- 33
4.4 Performance & indexation----------33
4.5 Backend : filtres et recherche---------- 34
4.6 Retour critique & questions---------- 34
Mission 5 – Système de traçabilité---------- 35
5.1 Modèle de données des logs----------35
Mission 6 – Évolutions fonctionnelles et optimisation technique---------- 38
Contexte initial---------- 38
Enrichissement de l'IHM et adaptation des routes----------39
Extension des critères de recherche---------- 39
Optimisation des performances et fiabilité----------40
Coordination, recette et déploiement----------40
II. Valorisation des compétences----------41
Partie 1 — Compréhension et cadrage des besoins métiers----------41
1.1 Contexte global des trois projets IA & Data---------- 41
1.2 Recueil et formalisation du besoin (BC04)----------41
1.3 Analyse des sources et contraintes techniques (BC01)----------42
1.4 Qualification et tri des données (BC01)---------- 43
1.5 Retour critique sur le cadrage (BC05)---------- 43
Partie 2 — Conception, entraînement et évaluation de la solution IA----------43
2.1 Du corpus brut à l'entrée réseau (BC02)----------44
2.2 Choix des architectures et justification (BC02)---------- 44
2.3 Stratégie d'entraînement et paramètres clés (BC02)---------- 45
2.4 Évaluation détaillée et interprétabilité (BC02)----------45
2.5 Intégration dans une API et perspectives de production (BC03)----------46
Partie 3 — Gestion des activités et tâches du projet IA----------46
1. Méthodologie de pilotage (Kanban & Gantt) (BC04)----------46

2. Répartition des responsabilités et montée en compétence (BC04)----------47
3. Gestion des imprévus (BC04)---------- 48
Partie 4 – Communication et veille technologique---------- 48
1. Communication interne et documentation (BC05)----------48
2. Veille technologique (BC05)----------48
3. Amélioration continue (BC05)----------49
III. Conclusion et Perspectives----------49
Perspectives pour la Métropole de Montpellier :----------49
Axes d'amélioration :----------49
Apports professionnels et personnels :----------50
Annexes----------50

Introduction

1. Remerciements

Pour commencer, j'aimerais remercier toutes les personnes qui m'ont accueilli et accompagné lors de ce stage.
Je remercie tout d'abord très fortement mon tuteur de stage, Mohamed Achbani, et Frédéric Fayolle pour leur confiance, leur disponibilité et leur accueil au sein de la métropole de Montpellier.
Je remercie également l'ensemble de l'unité projet et développement pour leur accueil bienveillant, leur aide et les échanges professionnels et non professionnels tout au long du stage.
Cette expérience m'a permis de découvrir un milieu professionnel complet et complexe et a renforcé ma motivation à poursuivre dans ce domaine.

I. Environnement professionnel

1. Présentation de la métropole

Historique et but de la Métropole de Montpellier

Créée officiellement en 2015 , la Métropole de Montpellier (MMM : Montpellier Méditerranée Métropole) est une intercommunalité française regroupant 31 communes du département de l'Hérault en région Occitanie. Elle est née de la transformation de l'ancienne communauté d'agglomération de Montpellier dans le but de répondre aux nouvelles demandes territoriales, de regrouper les ressources et d'optimiser les services publics.
La Métropole joue un rôle clé dans la gestion de multiples compétences transférées par ses communes membres : urbanisme, logement, développement économique, mobilité, environnement, transition énergétique, culture, numérique, etc.
Sa mission est double :

- Offrir un service public de qualité à l'échelle de toute l'agglomération,
- Préparer l'avenir du territoire grâce à une politique de développement durable, de numérique, et d'inclusion.

Son siège est situé à place Zeus de Montpellier.

Chiffres clés

La Métropole est structurée en pôles thématiques qui regroupent les différentes directions métier. Mon stage s'est déroulé dans le Pôle Numérique et Donnée (PND), qui regroupe :

- 110 agents environ, répartis autour de 6 services.
- Un système d'information transversal pour l'ensemble des services métropolitains.
- Une organisation structurée autour de plusieurs services métiers SI.

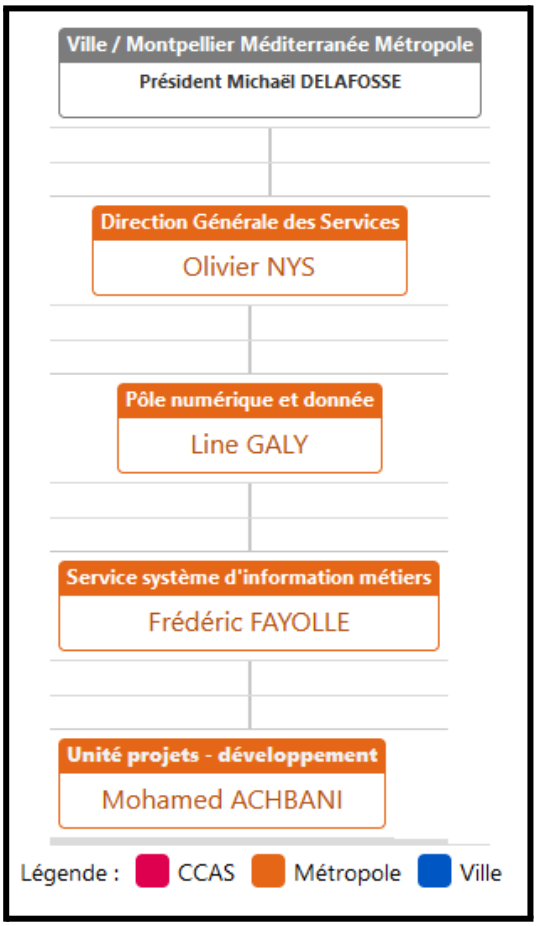
Le Pôle a pour but d'assurer ces 6 principes : (d'après l'intranet de la MMM)

- Faciliter et innover dans les accès à l'information des citoyens et partenaires (e-services prioritaires, fracture numérique, stratégie web, etc.) ;
- Apporter une clarification rapide de l'organisation du Système d'information (SI) afin de gagner en lisibilité (attentes fortes sur l'accompagnement aux usages et l'harmonisation de processus afin de faciliter la mutualisation d'ensemble) ;
- Améliorer la couverture applicative du SI et les usages (domaines fonctionnels non couverts par des outils et modules spécifiques, flexibilité des outils) ;
- Structurer la stratégie et le management de la donnée (valorisation des référentiels existants, mise en place de tableaux de bord, sécurisation du capital de données, exploitation, etc.) ;
- Favoriser et améliorer la transversalité au sein des collectivités (pratiques collaboratives, partage de documents, processus transversaux, etc.) ;
- Finaliser et accompagner le nomadisme (PC portables, travail à distance, etc.).

Organigramme du PND et du SSIM

Le Pôle Numérique et Données est structuré en plusieurs services. Mon projet a été réalisé dans le Service Systèmes d'Information Métier (SSIM), qui s'occupe de la conception, le développement et la maintenance des applications aussi bien pour des logiciels du marché que du développement sur mesure (interne et externe).
Dans ce même service, j'ai été rattaché à l'Unité Projet & Développement, où j'ai été intégré en tant que stagiaire développeur fullstack.

Organigramme de l'organisation de la ville / Métropole Méditerranée Métropole
(Centré sur mon unité) provient de l'intranet Montpellier.fr

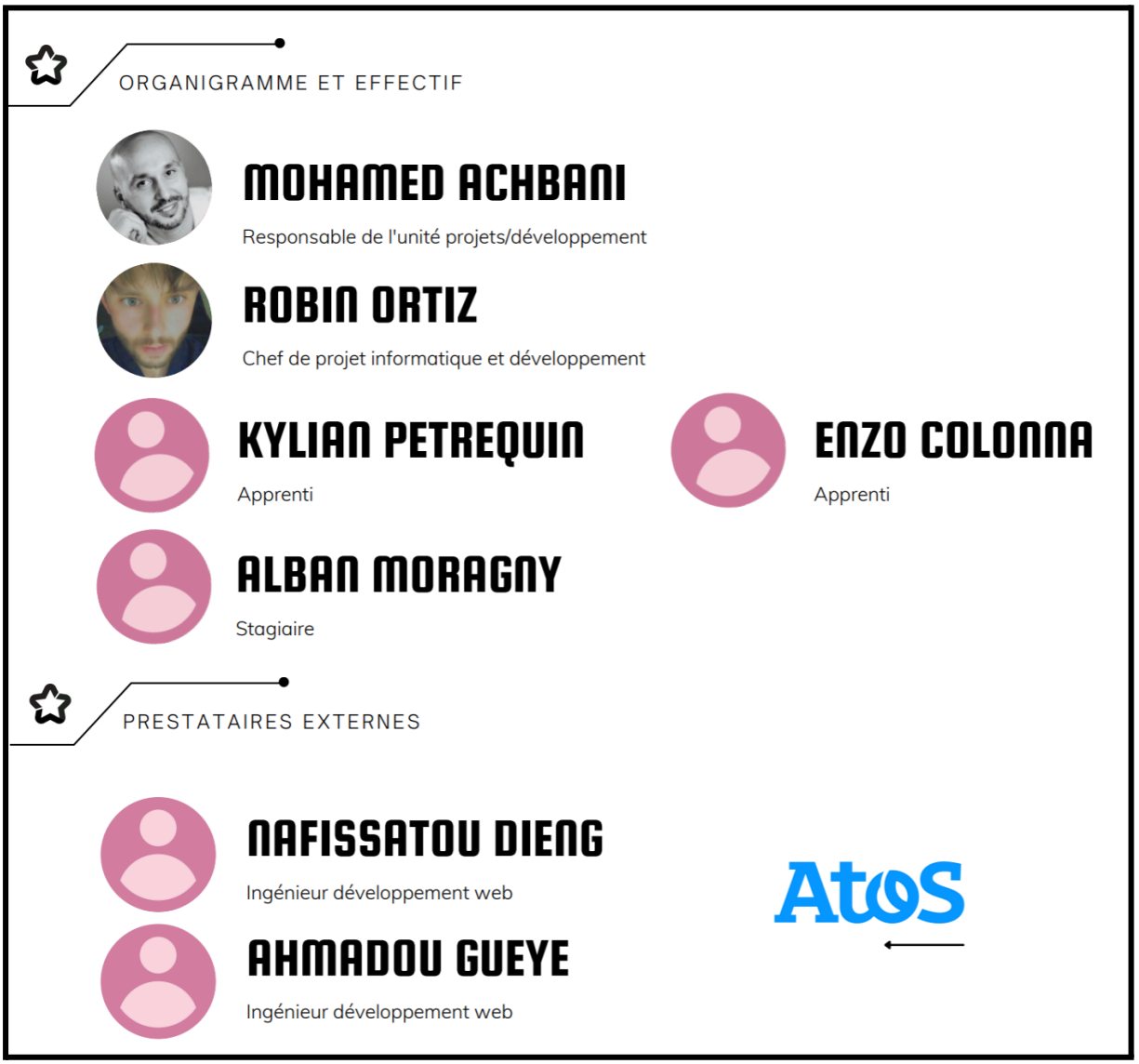


2. Positionnement du stage

L'Unité Projet & Développement

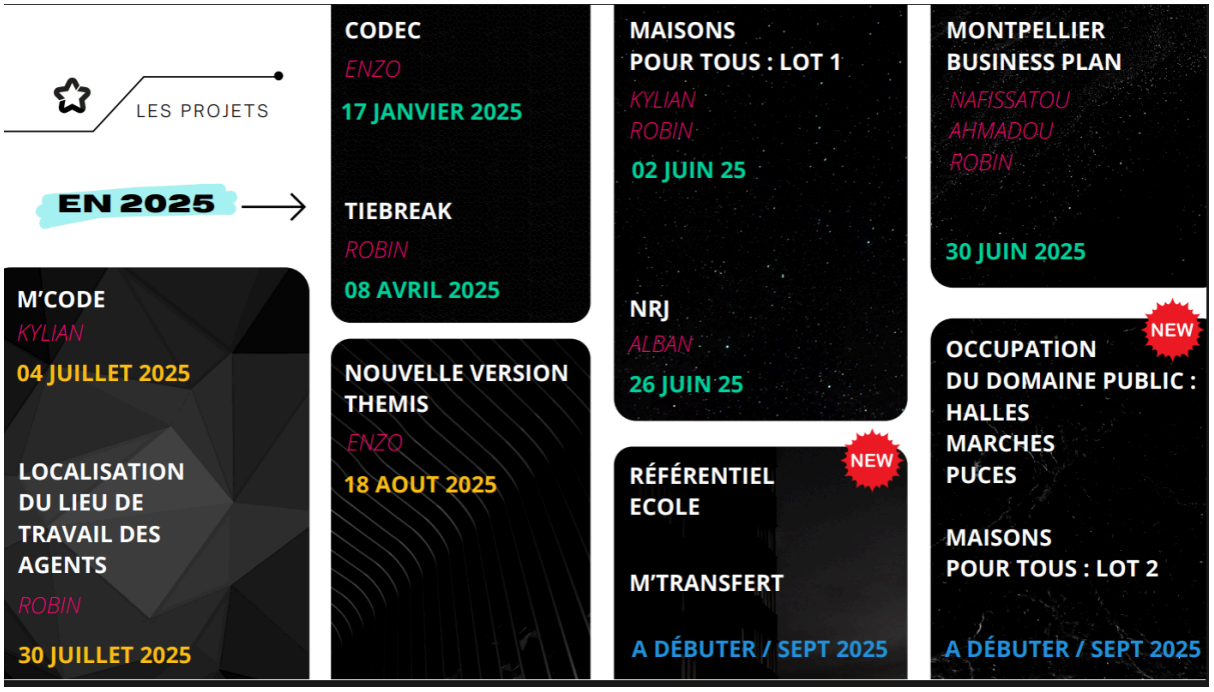
Cette unité est chargée de répondre aux besoins spécifiques de développement d'applications métiers au sein de la Métropole et de la ville notamment pour les différents pôles métiers. Elle intervient depuis l'analyse des besoins jusqu'au déploiement des solutions, en passant par le maquettage, le développement, les tests, l'intégration continue et la documentation.
J'ai intégré cette unité pour participer au développement d'un outil web de gestion de facture énergétique à destination du Pôle Sobriété Énergétique, en collaboration avec d'autres services techniques.

Organigramme de l'organisation de l'unité Projet & Développement
(Réalisé pour une présentation lors d'une réunion interne)



Contexte de mon équipe

Durant mon stage, notre unité était organisée par projet, chaque membre étant associé à un projet spécifique avec des objectifs précis. Nous étions tous supervisés par notre responsable. Voici un exemple de répartition des projets :



Étant donné que les technologies utilisées étaient plus ou moins similaires d'un projet à l'autre, la collaboration entre les projets était grandement facilitée.

Transversalité du stage

Le projet m'a conduit à travailler avec plusieurs pôles et directions, notamment :

- Le Pôle Sobriété Énergétique, dirigé par Mme Caroline CANALE, en charge du pilotage de la consommation énergétique des bâtiments publics.

- Le Service Politique et Valorisation de la Donnée, dirigé par M. Frédéric SAUNIER, gestionnaire de la stratégie data et de l'intégrité des flux d'informations.

Cette obligation de transversalité m'a permis de développer une vision globale des nécessités du pôle énergétique et numérique, tout en interagissant avec des futurs utilisateurs, des référents métiers et des responsables techniques.

Mon rôle au sein de l'équipe

En tant que stagiaire développeur fullstack, mes missions ont été les suivantes :

- Analyse des besoins métier et conception de l'architecture de l'outil.
- Développement d'une application web sécurisée (Django / React) pour la gestion des factures et consommations d'énergie.
- Implémentation d'un système de traçabilité et de gestion des utilisateurs.

- Participation aux réunions de suivi et à la documentation du projet.

Durant ce stage, j'ai fait preuve d'autonomie, de rigueur et d'esprit critique, en proposant des améliorations sur l'organisation des données, l'optimisation des requêtes SQL et sur l'UI & UX de l'application.

3. Contexte du projet

Équipe du projet

CANALE Caroline - Responsable Service Etudes et Gestion des Fluides
MLADENOVIC Slavko - Contrôleur de gestion interne
OLIVIER Nicolas - Chargé de Gestion Énergie & VASSEUR Simon - Stagiaire
SAUNIER Frédéric - Chef de projet décisionnel
MORAGNY Alban - stagiaire développeur FullStack PND

Limites des fichiers Excel précédemment utilisés

Avant le début du projet, la gestion des factures énergétiques à la Métropole de Montpellier reposait principalement sur des fichiers Excel partagés. Ce système, bien que accessible à tous et simple à prendre en main, présentait rapidement de sérieux problèmes dans un contexte de gestion à grande échelle :

- Manque de centralisation : chaque agent ou service gérait ses propres fichiers, ce qui entraînait des doublons, des versions contradictoires ou dépassées, sans véritable coordination.

- Erreurs humaines fréquentes : sans automatisation ni validation, des erreurs de saisie ou de calcul passent facilement inaperçues.

- Aucune traçabilité : il était impossible de savoir qui avait modifié quoi, ni quand, ce qui complique la gestion et peut notamment conduire sur des doublons de modifications de la même facture.

- Exploitation compliquée des données : produire des statistiques fiables (consommation, alertes, surcoûts...) demandait beaucoup de temps et de traitements manuels.

- Sécurité insuffisante : les fichiers étaient souvent partagés par mail ou sur team's, sans réel chiffrement, ce qui exposait les données à des risques de fuite ou de corruption.

Ces problèmes nuisent directement au bon fonctionnement du Pôle Sobriété Énergétique, qui avait besoin d'un outil fiable, sécurisé et facile à utiliser au quotidien.

Histoire du projet

La limite des fichiers Excel a été signalée pour la première fois en mars 2023. À cette époque, le pôle de gestion avait soumis une demande d'évolution. Une fois cette demande acceptée une série de réunions de projet a été organisée pour définir les besoins du pôle énergie et déterminer les outils à utiliser étalé sur plusieurs mois.

En 2024, en raison d'un manque de personnel à associer au projet, celui-ci a été mis en suspens jusqu'au début de l'année 2025. C'est à ce moment-là que le pôle données et l'unité projet et développement ont relancé le projet avec l'équipe présentée précédemment. Après quelques réunions de projet, le travail a finalement commencé début 2025.

Objectifs métiers et enjeux liés à la donnée

Face à ces constats, un besoin s'est affirmé de concevoir une application web centralisée capable de répondre aux enjeux métiers suivants :

- Fiabilité des données : la cohérence et l'uniformité des informations de facturation et de consommation doivent être garanties.
- Accessibilité multi-acteurs : l'outil doit être accessible par différents profils (agents techniques, décideurs, analystes) avec un système de droits adaptés.
- Automatisation des tâches répétitives : l'importation de données énergétiques, la génération d'indicateurs ou les vérifications manuelles doivent être en partie automatisées.
- Exploitation décisionnelle : à terme, la donnée énergétique doit pouvoir être utilisée pour établir des préconisations, identifier des anomalies ou optimiser les consommations.
- Traçabilité et suivi : chaque action réalisée par un utilisateur devrait pouvoir être retrouvée dans un historique, afin de faciliter le suivi, corriger d'éventuelles erreurs ou répondre à des besoins d'audit interne.

Ce projet s'inscrit pleinement dans une logique de transition numérique, à la croisée entre la gestion énergétique et la maîtrise des données publiques.

4. Objectifs du projet

Le projet mené pendant mon stage a eu pour objectif principal de concevoir et développer une application métier destinée à remplacer les fichiers Excel existants, avec une attention portée à la sécurité, à la traçabilité et à l'efficacité de gestion. Les objectifs techniques et fonctionnels définis dès le lancement du projet étaient les suivants :

Authentification sécurisée et gestion des accès

- Mise en place d'un système d'identification fiable (JsonWebToken), avec gestion des rôles et droits d'accès par utilisateur.

Consultation et modification des informations énergétiques

- Interface intuitive permettant d'afficher, filtrer, modifier et historiser les factures et consommations liées à chaque point de livraison (PDL).

Traçabilité complète des modifications

- Enregistrement de chaque action utilisateur (création, modification, suppression), associé à un système de logs horodatés.

Préparation à la scalabilité

- Base de données relationnelle optimisée (PostgreSQL), structuration des modèles pour permettre une visualisation sur des outils tels que Business Object.

5. Cartographie du système d'information

Le projet a nécessité la mise en place d'une architecture technique sécurisée et adaptée aux contraintes d'une collectivité territoriale. La cartographie du système d'information ci-dessous détaille les composants techniques, les acteurs impliqués et les flux de données échangés au sein de l'application.

---

1. Architecture technique

L'architecture de la solution suit un modèle client/serveur découpé en trois couches principales :

- Frontend React : assure l'affichage des interfaces utilisateur, la navigation, les formulaires, les appels API.

- Backend Django REST : gère les règles métier, les droits d'accès, les traitements, les logs.

- PostgreSQL : stocke les données normalisées et structurées (factures, PDL, logs...).

L'ensemble est conteneurisé via Docker et versionné via GitLab, avec intégration continue. Suite à cela, on le déploie sur

2. Schéma réseau simplifié (déploiement en interne)

```
[Utilisateur] ── HTTPS ──> [Reverse Proxy] ──> [Serveur Django/API]
                                  │
                                  └──> [Base PostgreSQL] (accès local)
```

Le système est prévu pour une exploitation interne au réseau de la Métropole. Des règles de pare-feu et d'authentification restreignent les accès au SI internes uniquement.

3. Utilisateurs et rôles

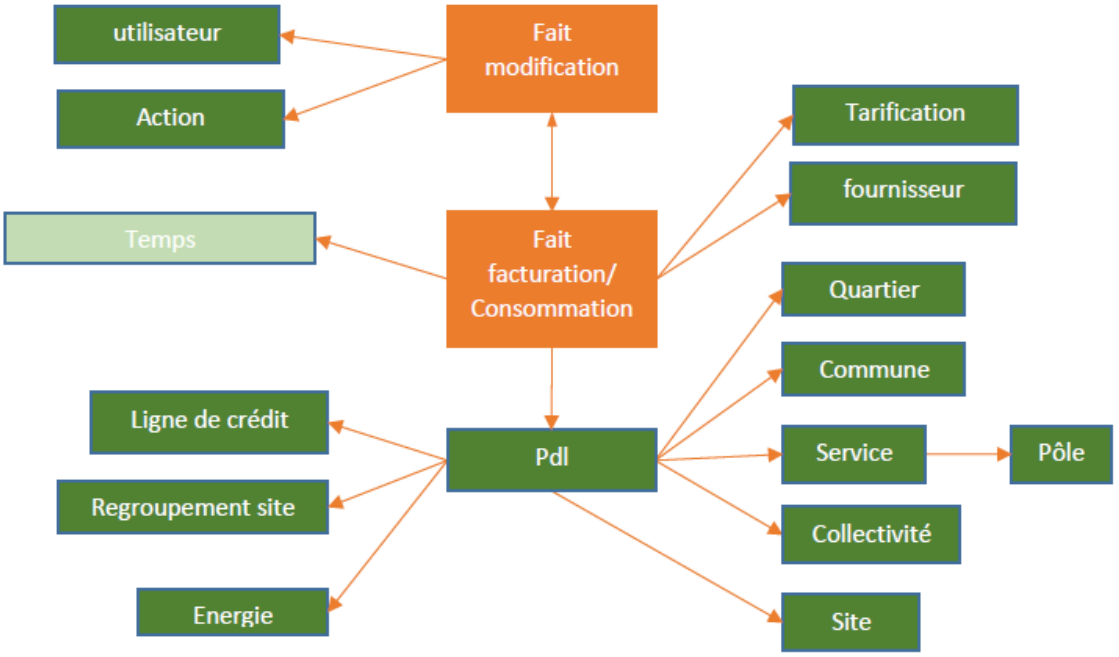
Rôle	Accès autorisés	Objectif
Lecteur	Visualisation globale, export, supervision	Suivi et mise à jour des données
Modificateur	Visualisation + modification, ajout, suppression des factures et logs	Analyse, reporting, traçabilité, correction
Administrateur	Gestion des comptes + identique à modificateur	Maintien de la sécurité

4. Données manipulées (tables principales)

Table	Description
fait_facturation	Contient toutes les informations des factures : numéro, PDL, date, montant…

dim_pdl	Référentiel des points de livraison (PDL) : identifiant, adresse, etc.
dim_energie	Liste des types d'énergie utilisés (eau, élec, gaz, etc.)
dim_tarification	Référentiel des types de tarification associés aux factures
utilisateurs	Informations des utilisateurs, rôles, identifiants
fait_modifications	Enregistre toutes les actions (create/update/delete) sur les factures

5. Schéma relationnel simplifié



Hormis les dimensions liées à la facturation (temps, tarification et fournisseur, la modélisation est faite sur une seule dimension PDL afin de ne pas gérer les évolutions des branches rattachées à cette dimension alors que les faits seront ajoutés au fil du temps.

6. Technologies et méthodologies employées

1. Technologies principales

Le projet s'appuie sur une stack FullStack, combinant des technologies modernes et open source, bien adaptées aux environnements de développement et cohérent avec les autres applications développées par mon équipe.

Backend : Django REST Framework (Python)

Le framework Django a été utilisé pour :

- Structurer l'API RESTful (Models, ViewSet, Serializers, Permissions)
- Gérer la logique métier et les règles d'accès
- Implémenter la traçabilité avec les LogModifications
- Optimiser les accès aux données (requêtes filtrées, pagination, performance PostgreSQL)

Frontend : React.js

Le frontend a été conçu en React avec :

- Un design responsive basé sur MUI + composants personnalisés (SoftBox, SoftTypography…)
- L'intégration des appels API via axios
- Des vues dynamiques : tableau de bord, création/modification de factures, consultation des logs
- Des effets de transitions fluides avec framer-motion

Base de données : PostgreSQL

PostgreSQL a été utilisé comme base de donnée principal :

- Tables relationnelles normalisées (fait_facturation, dim_pdl, log_modifications…)
- Contraintes d'unicité et relations inter-tables (clé étrangère)
- Optimisation des requêtes complexes grâce aux index et jointures

Environnement : Docker

L'environnement de développement a été conteneurisé avec Docker :

- Conteneurs distincts pour le backend, frontend, et PostgreSQL
- Fichiers docker-compose.yml utilisés pour le déploiement local rapide
- Cohérence entre les environnements de dev/test/prod

Suivi du projet : GitLab

GitLab a été utilisé pour :

- Le versionning du code source (branches, merges)
- La gestion des issues et des objectifs
- La mise en place de pipelines CI/CD

---

2. GitLab CI/CD

Un pipeline GitLab CI/CD a été mis en place pour automatiser les étapes critiques du cycle de développement :

Étape	Description
Linting	Analyse du code pour détecter les erreurs de style ou de syntaxe
Tests	Lancement automatique des tests unitaires (backend principalement)
Build	Compilation du frontend React (mode test ou production)
Déploiement	Copie automatique sur un environnement distant (sur Portainer)

II. Valorisation des compétences

Partie 1 - En entreprise

Mission 1 – Analyse des besoins et maquettage

Bloc de compétences : Analyse des besoins et maquettage
Activité : Analyse des besoins et conception fonctionnelle
Compétence choisie : Recueil et analyse des besoins, conception fonctionnelle et prototypage

Contexte initial

Le projet s'inscrit dans une logique de modernisation du pilotage énergétique de la Métropole et de la Ville de Montpellier. Cette initiative est en lien direct avec les ambitions de sobriété énergétique et de valorisation de la donnée publique. Le système historique, basé sur des fichiers Excel partagés et retraités manuellement, ne permettait ni une traçabilité fiable des modifications, ni une gestion efficace du

volume croissant des données. Ce besoin de modernisation a été renforcé par l'intégration dans l'écosystème DigDash / Datamart PostgreSQL, tel que décrit dans les spécifications fonctionnelles détaillées SFD Dataviz PISE Énergie.

Recueil des besoins

Dès les premières semaines, j'ai participé à plusieurs réunions de cadrage avec les représentants des pôles PISE, PND, SPVD et UOVD. Lorsque je n'étais pas présent physiquement, un compte rendu détaillé m'était systématiquement transmis. Cela m'a permis de comprendre les besoins spécifiques du service énergie, tels que l'intégration des données Astech et l'ajout des dimensions Surface et Collectivité dans l'univers BO. J'ai également pu cibler les utilisateurs finaux du système, notamment les analystes, agents et référents techniques, et identifier les flux de données prioritaires à intégrer, comme les factures de 2021 à 2024, les PDL et les tarifications.
En parallèle, j'ai effectué une analyse comparative entre les fichiers Excel existants, tels que la base patrimoine et facturation.xlsx, et le modèle cible décrit dans le Datamart NRJ. Cette analyse m'a permis de comprendre les divergences et d'identifier les zones à modéliser de façon plus robuste, en utilisant une clé fonctionnelle composée de facture_numero, pdl_id et fact_date (puisfact_conso_debut).

Contexte de réalisation

La réalisation de cette mission a impliqué plusieurs étapes clés. Tout d'abord, j'ai participé à des réunions de cadrage avec les représentants des différents pôles pour recueillir les besoins et comprendre les attentes.
Pour la conception fonctionnelle, j'ai proposé une cartographie fonctionnelle métier, validée par mon responsable de stage et l'équipe. Cette cartographie incluait le parcours utilisateur depuis la page d'accueil jusqu'à la modification d'une facture, les événements déclencheurs de logs (ajout, modification, suppression) et la gestion des rôles avec des droits différenciés selon les profils. J'ai également produit un tableau de synthèse des fonctionnalités, incluant l'ajout et la modification de factures, la visualisation des logs, les filtres dynamiques (par pôle, collectivité, année, site, etc.) et l'accès aux documents PDF liés.

Contraintes réglementaires, techniques ou administratives

Le projet devait respecter plusieurs contraintes, notamment l'intégration dans l'écosystème DigDash / Datamart PostgreSQL et les spécifications fonctionnelles détaillées. Il était également nécessaire de gérer les rôles et les droits différenciés selon les profils des utilisateurs.

Ressources utilisées

Humaines : La collaboration avec les représentants des pôles PISE, PND, SPVD et UOVD a été essentielle pour recueillir les besoins et valider les différentes étapes du projet.
Matérielles : J'ai utilisé divers outils pour mener à bien cette mission, notamment des fichiers Excel pour l'analyse comparative, des outils de cartographie fonctionnelle pour la conception, et Trello pour la gestion de projet. Pour le développement frontend, j'ai utilisé des composants MUI + Soft.
Méthodologiques : Les réunions de cadrage, l'analyse comparative, la cartographie fonctionnelle et la gestion de projet avec Trello ont été les principales méthodes utilisées. Ces méthodes ont permis de structurer le projet et de suivre l'avancement en temps réel.

Possibilité d'exercice à distance

Oui, une partie du travail a été réalisée à distance, avec la transmission de comptes rendus détaillés pour assurer la continuité et la cohérence du projet.

Évolutions majeures

Cette compétence a été traversée par des évolutions majeures ces 10 dernières années, notamment l'adoption de systèmes de gestion de données plus robustes, l'utilisation de méthodes agiles pour la gestion de projet, et l'intégration de technologies modernes pour le frontend comme React et MUI.

Évolutions futures

Les évolutions futures pourraient inclure l'adoption de nouvelles technologies pour l'analyse de données et l'intelligence artificielle, l'automatisation accrue des processus de collecte et d'analyse des données, et l'intégration de nouvelles réglementations sur la gestion des données publiques.

Acquisition

Cette mission m'a permis d'acquérir plusieurs compétences professionnelles et humaines. J'ai développé une compréhension approfondie des besoins spécifiques du service énergie et des utilisateurs finaux. J'ai également appris à travailler en collaboration avec différentes entités et à clarifier les demandes par écrit. Enfin, j'ai acquis des compétences en gestion de projet et en utilisation d'outils comme Trello.

Difficultés rencontrées

L'une des principales difficultés a été le temps nécessaire à une bonne formalisation du besoin. La coexistence de plusieurs entités (PISE, PND, PFCG, etc.) a rendu cette tâche complexe. Pour surmonter cette difficulté, j'ai clarifié chaque demande

par écrit au travers de rapports et réalisé un travail de vulgarisation continu pour expliquer les contraintes techniques aux métiers et vice versa. J'ai également travaillé en décalé avec des comptes rendus lorsque je n'étais pas en réunion.

Pistes de progression

Pour progresser, je compte continuer à développer mes compétences en gestion de projet et en analyse des besoins. Je souhaite également approfondir mes connaissances en technologies modernes pour le frontend et l'analyse de données. Enfin, j'aimerais améliorer mes compétences en communication et en collaboration avec différentes entités.

Retour critique

Cette mission m'a confronté à une réalité peu abordée en formation : le temps nécessaire à une bonne formalisation du besoin. Le fait que plusieurs entités coexistent (PISE, PND, PFCG, etc.) m'a poussé à clarifier chaque demande par écrit au travers de rapports et à réaliser un travail de vulgarisation continu. Travailler en décalé avec des comptes rendus lorsque je n'étais pas en réunion a également été une expérience enrichissante. Le choix de prioriser l'UX dès le début, plutôt que de coder directement, a été payant : cela a fluidifié toute la chaîne du projet, jusqu'aux livraisons finales.
En conclusion, cette mission a été une expérience très enrichissante qui m'a permis de développer plusieurs compétences professionnelles et humaines. Elle m'a également donné l'opportunité de travailler sur un projet concret et impactant pour la Métropole et la Ville de Montpellier.

Mission 2 – Authentification & Sécurité

Introduction

Sécuriser l’accès à l’application de pilotage énergétique a été un point central dès la phase de conception, pour garantir :

- La confidentialité des données sensibles (factures, PDL, logs),
- La traçabilité des actions utilisateurs,
- La conformité avec l’annuaire central de la Métropole (SSO et API 3M).

Depuis février 2025, nous avons donc implémenté un mécanisme d'authentification reposant sur l'API centrale (api3m_url) de la Métropole et les standards SSO, intégré au sein de notre backend Django REST.

---

1. Authentification via api3m_url

a. Flux « login/password »

1. Formulaire de connexion (React)
   L'utilisateur saisit son identifiant (numéro de fonction ou login Métropole) et son mot de passe.

2. Requête au backend (Django)
   Le frontend envoie un POST /api/auth/login/ contenant { username, password}.

3. Proxy vers api3m_url
   Django relaie ces informations à l'API centrale (https://api3m.metropole.fr/auth/token/), qui renvoie :

   - Un access_token (durée courte, p.ex. 30 mn),
   - Un refresh_token (durée plus longue),
   - Les attributs utilisateur (nom, email, rôles attribués; immatriculation…).

4. Stockage sécurisé

   - Le access_token est stocké en mémoire côté client (React state ou contexte) et jamais en localStorage.
   - Le refresh_token est conservé en HttpOnly cookie (propriété sécurisée, inaccessible au JS).

5. Réponse au frontend
   Django renvoie un statut 200 + les métadonnées utiles (rôles, expiration).

b. Flux SSO (Single Sign-On)

- Le bouton « Connexion Métropole » redirige vers un endpoint Django /api/auth/sso/redirect/.

- Django renvoie une redirection HTTP 302 vers l'URL SSO de l'API 3M, avec un callback sur /api/auth/sso/callback/.

- Après authentification sur le portail Métropole, l'utilisateur est renvoyé sur notre callback, où :

    1. Django récupère le code OAuth2.
    2. Échange ce code contre un JWT auprès de api3m_url.
    3. Calque exactement le même stockage de tokens que pour le login/password.

---

2. Séparation frontend / backend

Composant	Rôle
React SPA	Auth formulaire, gestion du token en mémoire, redirections
Django API	Proxy auth, validation des tokens, enforcement RBAC
PostgreSQL	Stockage des données applicatives, pas des tokens

Chaque appel React → Django inclut le header Authorization: Bearer <access_token>.

Les CORS sont strictement configurés pour n'autoriser que notre domaine front.
Les containers Docker séparent physiquement l'API auth de l'API métier.

3. Gestion des rôles et autorisations

Nous avons défini une hiérarchie de trois rôles pour contrôler l'accès aux fonctionnalités :

- Utilisateur (niveau 1)

- Auditeur (niveau 2)

- Admin (niveau 3)

Côté frontend, avant chaque route protégée, on :

1. Vérifie la présence d'un access token.

2. Interroge /api/auth/user pour récupérer user.role.

3. Compare roleHierarchy[user.role] vs roleHierarchy[requiredRole].

4. Re-dirige automatiquement vers /unauthorized si insuffisant.

L'attribution des rôles se fait via une interface d'administration. Un simple panneau permet de changer le rôle d'un utilisateur, ce qui déclenche un appel PATCH au backend et consigne l'action dans la table auth_logs.

4. Cycle de vie des tokens & Interceptors Axios

Pour sécuriser chaque appel API, nous utilisons deux interceptors Axios :

1. Request interceptor
    - Récupère le accessToken (stocké en mémoire).
    - Vérifie l'expiration (via son exp décodé).
    - Si expiré, appelle l'endpoint /token/refresh/ avec le refreshToken (cookie HttpOnly) pour obtenir un nouvel accessToken.
    - Injecte ensuite Authorization: Bearer <token> dans l'en-tête.
2. Response interceptor
    - Si le serveur renvoie 401 ou 403, purge tous les tokens et redirige vers /login.
    - Pour une erreur réseau/CORS, fait de même pour éviter une situation bloquante.

Cette configuration assure une session utilisateur fluide : le renouvellement et les blocages sont entièrement invisibles à l'utilisateur.

5. Tests et validations

Afin de garantir la stabilité de la solution sans impacter la production, nous avons mis en place une chaîne de validation progressive, de l'environnement local jusqu'à la préproduction CI/CD.

5.1 Environnement local

- Installation et tests « à blanc »
  Chaque développeur clone le dépôt GitLab et lance les services via docker-compose up --build, pointant par défaut vers une base PostgreSQL de test nommée nrj_test.

- Vérifications manuelles

  - Création / modification / suppression de factures et PDL.
  - Navigation dans l'IHM, affichage des logs et contrôles de rôles.
  - Validation du cycle d'authentification (login/password, refresh token).

Cette phase nous permet de valider chaque changement en isolation, directement sur nos postes, sans risque pour la prod.

5.2 Pipeline CI/CD (GitLab)

Sur chaque push vers les branches concernées, quatre jobs clés sont déclenchés :

1. build-staging

   - Construction de l'image Docker de l'API et de celle du frontend.

   - Publication dans le registry interne (registry.metropole.mtp/nrj:${CI_COMMIT_SHORT_SHA}).

2. staging

   - Déploiement de l'image build-staging sur l'environnement de préproduction.

   - Validation basique du container en santé (/health/).

Les deux autres jobs build-production et production sont définis mais non exécutés dans la phase de tests.

5.3 Base de données de test

- La copie du schéma de production dans nrj_test est assurée par le service base de données de la Métropole : je n'interviens pas directement sur ce processus.

- La base nrj_test est automatiquement réinitialisée à chaque exécution du pipeline staging, garantissant un état vierge pour chaque déploiement.

5.4 Recette manuelle en préproduction

Après succès des jobs build-staging et staging :

1. Je me connecte à l'URL de préproduction (p. ex. https://staging.nrj.metropole.mtp).

2. Authentification

    - Test du login/password, du refresh token et du SSO via api3m_url.

3. Fonctionnalités critiques

    - Création/modification/suppression de factures ;

    - Contrôles de la hiérarchie de rôles (Utilisateur, Auditeur, Admin) ;

    - Vérification de la bonne génération des logs d'historique.

5.5 Retour critique

- Points forts

    - Pipeline CI/CD court et fiable : build + staging en moins de 3 minutes.

    - Base de test gérée par la DSI, ce qui allège la charge de maintenance côté dev.

    - Recette manuelle en préprod très représentative de la prod réelle.

- Axes d'amélioration
    - Mettre en place des tests unitaires Django et React pour couvrir au moins les endpoints critiques.
    - Automatiser des vérifications de santé plus complètes (smoke tests) après déploiement en staging.
    - Documenter le process CI/CD dans le README pour onboarder rapidement de nouveaux contributeurs.

Avec cette organisation, chaque évolution est d'abord validée localement, puis automatiquement packagée et déployée en staging, avant une dernière phase de recette manuelle qui garantit la cohérence fonctionnelle et la stabilité de la solution.

6. Retour critique

Points forts

- Intégration transparente avec l'annuaire Métropole (API 3M + SSO).
- Hiérarchie de rôles simple et facile à administrer.
- Gestion automatique de la durée de vie des tokens pour l'utilisateur.

Axes d'amélioration

- Ajouter des tests de sécurité automatisés (analyse statique, pentests légers).
- Enrichir les logs (user agent, localisation approximative).
- Documenter l'API d'authentification avec OpenAPI/Swagger pour faciliter la maintenance.

Mission 3 – Interface utilisateur

Cette mission porte sur la conception et le développement de l'Interface Homme-Machine (IHM) de l'application de pilotage énergétique, en s'appuyant sur les composants SoftUI, un design responsive, et l'intégration directe des appels API définis dans notre projet Métropole de Montpellier.

3.1 Structure des routes et pages

Le routing de l'application est déclaré dans src/routes/index.js et détermine les pages accessibles selon les rôles :

Route	Composant	Rôle requis
/login/	<Login />	aucun
/rgaa/	<RgaaNonConforme />	aucun
/logout/	<Logout />	aucun
/callback/	<Callback />	aucun
/unauthorized	<Unauthorized />	aucun
/dashboard/	<Dashboard />	Utilisateur
/admin/	<Admin />	Admin
/invoice/list	<InvoiceList />	Utilisateur
/invoice/view/:fact_numero/:pdl_id/:fact_date	<InvoiceView />	Utilisateur
/invoice/modif/:fact_numero/:pdl_id/:fact_date	<ModifyInvoice />	Auditeur
/invoice/create	<CreateInvoice />	Auditeur

PrivateRoutes encapsule chaque route privée, vérifie le token et compare la hiérarchie des rôles (Utilisateur < Auditeur < Admin).

Les pages publiques (login, logout, callback, rgaa, unauthorized) utilisent le layout "none" pour masquer la navigation principale.

3.2 Pages principales

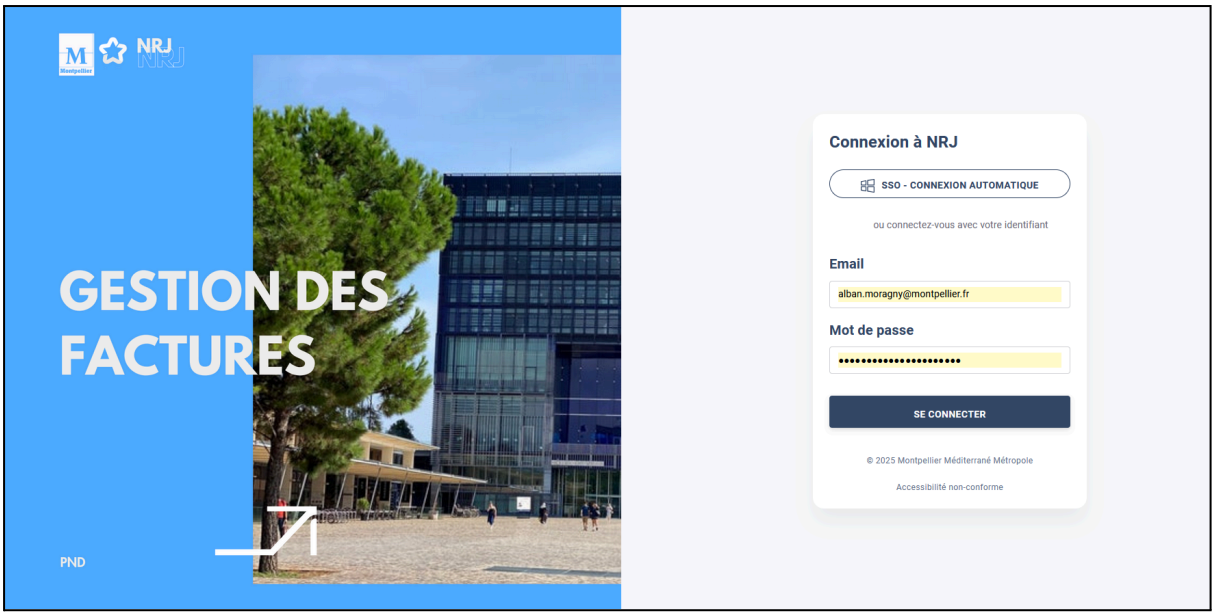
3.2.1 Login

- Composants :
  - SoftBox pour le conteneur,
  - SoftInput pour username/password,
  - SoftButton pour "Se connecter",

- SoftButton secondaire pour “Connexion Métropole” (OIDC).

- État/UX :
  - Après soumission, appel login(username, password) → réception { access, refresh, user }.
  - Sur erreur 401, affichage de SoftAlert rouge : “Identifiants invalides.”

Image de la page de connexion de l’application (version déployé)



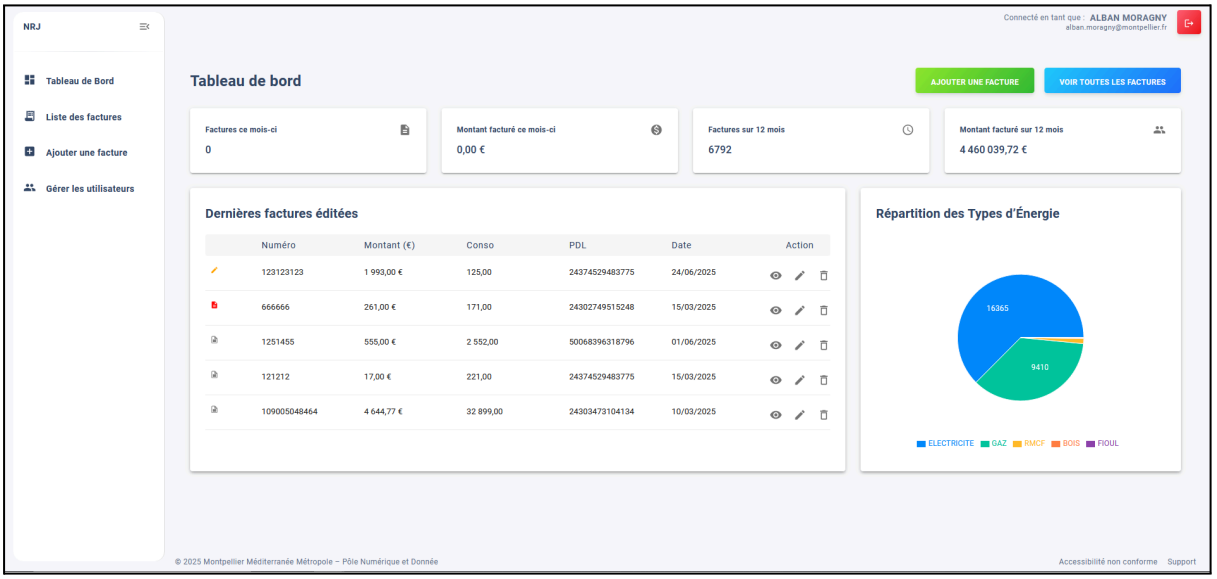
3.2.2 Dashboard

- Vignettes clés (getStatsDashboard()):
  - Nombre de factures ce mois, montant total, évolutions annuelles.

- Graphiques Camembert:
  - Principalement implémenté pour le design de la page plus que pour l’utilité.

- Liste rapide (getLatestFacturations()):
  - Tableau SoftTable des 5 dernières factures, lien vers la vue détaillée.

Image de la page du tableau de bord (version déployé)



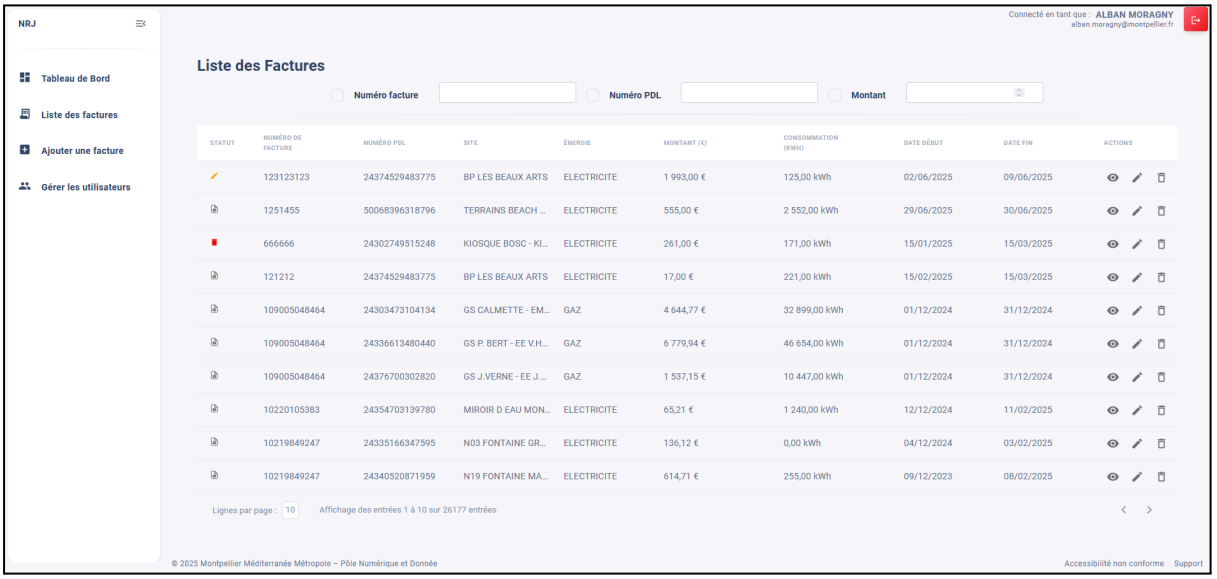
3.2.3 Liste des factures

- Endpoint : getFacturations({ limit, offset, filters }).

- Fonctionnalités :
  - Recherche par numéro, PDL, montant, tri (ordering).
  - Pagination manuelle (“Voir plus de résultats”).

- Composants SoftUI :
  - SoftSearchBar, SoftTable avec les colonnes Status, Numéro, PDL, Site, Energie, Montant, Consommation , date de début et de fin et Actions (View/Edit/Delete).
  - Bouton “Créer une facture” redirige vers /invoice/create.

Image de la page de la liste des factures (version déployé)

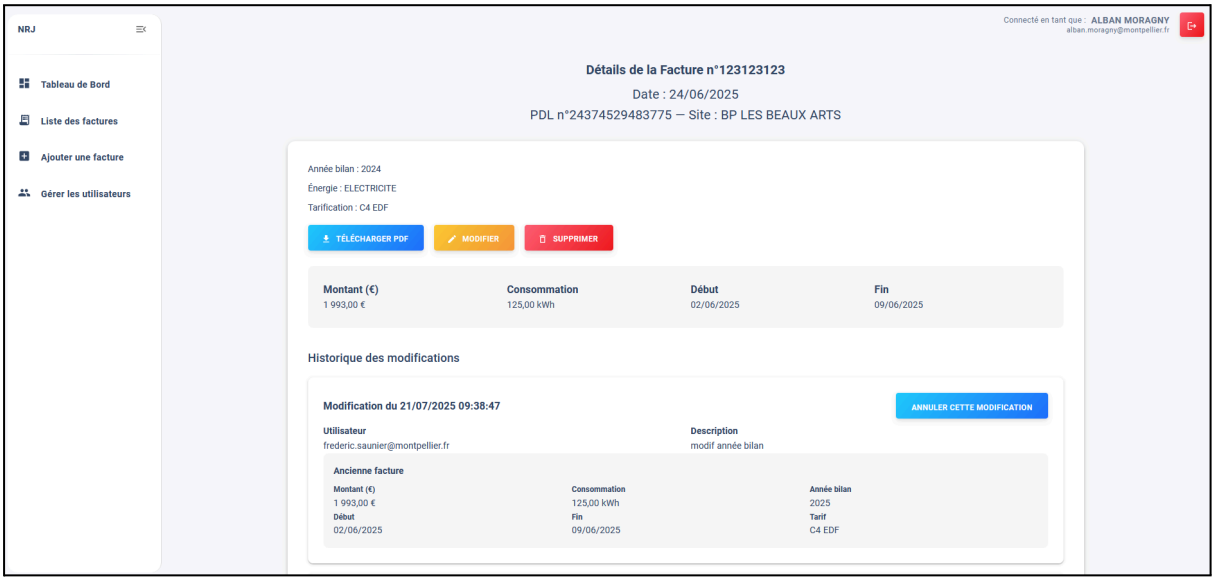


3.2.4 Vue d'une facture

- Endpoint : getFacturation(fact_numero, pdl_id, fact_date).

- Affichage :
  - Détails de la facture, PDF intégré via <object> ou <iframe>.
  - Boutons "Modifier" (si rôle ≥ Auditeur) et "Voir logs".

Image de la page d'une facture



3.2.5 Création / modification de facture

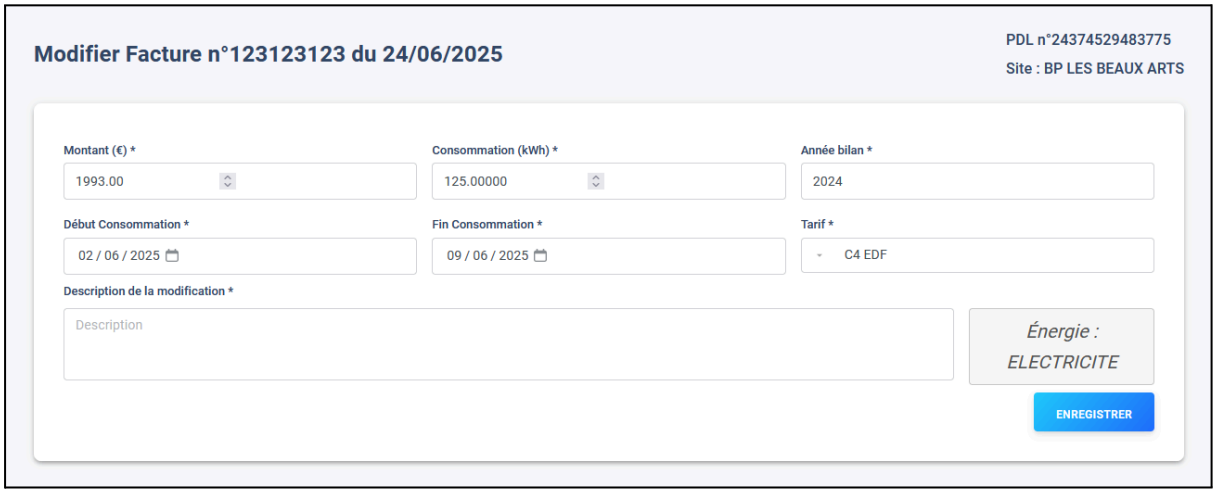
- Endpoints :
  - Création → addFacturation(data) POST /api/nrj/fait_facturation/

- Modification → updateFacturation(...) PATCH /api/nrj/fait_facturation/patch/.../

- Formulaire :
  - Champs contrôlés : Fact_numero, PDL (select alimenté par getPdlInfoByNumero), Date Picker, Montant, Type énergie (getEnergieTypes()), Tarification (getTarifications())
  - Upload PDF : wrapper SoftFileUpload
  - Validation : désactivation du bouton tant que !form.isValid

- UX :
  - Affichage SoftSpinner sur soumission
  - Redirection vers la liste ou la vue détaillée après succès

Image de la modification d'une facture



3.3 Composants SoftUI et responsive design

- SoftGrid & SoftBox :
  - Utilisés partout pour la structure, avec spacing={2} ou p={2}.

- SoftCard / SoftTypography :
  - Titres (variant="h6"), textes (variant="body2").

- Breakpoints (définis dans theme.js) :
  - Mobile (<600px) : colonnes full-width, menus masqués derrière un burger.
  - Tablette (600–960px) : drawer collapsible.
  - Desktop (>960px) : navigation latérale fixe, multi-colonnes.

- Tests manuels sur Chrome et Firefox :
    - iPhone 6 / Pixel 2 simulés, corrections sur marges et tailles de texte.

3.4 Intégration des appels API

Chaque page consomme directement les fonctions exposées dans services/api.js :

Fonction	**Usage**
me()	Récupération du profil pour AuthContext
getFacturations()	Liste & filtre de factures
getFacturation()	Détails pour /invoice/view
addFacturation()	Création /invoice/create
updateFacturation()	Modification /invoice/modif
delFacturation()	Suppression via bouton “Delete”
getFacturationsStats()	Indicateurs chiffrés
getStatsDashboard()	Vignettes du dashboard
getLatestFacturations()	Liste rapide des dernières factures
getEnergieTypes(), getTarifications()	Populations des selects

Gestion des états :

- Pour chaque appel, on utilise useState + useEffect.
- Un seul loader global (Soft Spinner) sur la page entière.
- SoftAlert pour chaque erreur renvoyée (catch).

3.5 Retours utilisateurs et pistes d’amélioration

- Feedbacks internes (exemple) :
    - Caroline Canale a souligné la clarté du dashboard.
    - Slavko Mladenovic a demandé une barre de progression pour l’upload PDF sur /invoice/create.

- Modifications suite aux retours :

- Ajout d'un SoftProgressBar pour les uploads > 2 Mo.
- Ajout de tooltips sur les icônes d'action (Edit, Delete) pour davantage de pédagogie.

Pistes d'évolution

1. Storybook pour documenter chaque composant SoftUI.
2. Tests de rendu (Jest + React Testing Library) sur les pages critiques (Login, InvoiceList).
3. Optimisation perf : code splitting par route (React.lazy & Suspense).
4. Accessibilité : audit rapide WCAG pour le contraste et la navigation clavier

Mission 4 – Traitement des données

Lors de cette phase, l'enjeu principal a été de concevoir et mettre en œuvre la structure physique de la base de données, d'assurer l'intégrité et la performance des accès, et de proposer un back-end capable de filtrer et rechercher rapidement et de manière précise dans les tables.

4.1 Modélisation & création des tables

4.1.1 Modèle conceptuel (MCD)

Nous avons repris le MCD validé en mission 1, avec les entités principales :

- Fait_Facturation (fact_facturation)
- Dim_PDL (dim_pdl)
- Dim_Energie (dim_energie)
- Dim_Tarification (dim_tarification)
- Utilisateur (utilisateurs)
- Log_Modifications (log_modifications)

Chaque Fait_Facturation est lié à un PDL, un type d'énergie et une tarification, et génère zéro ou plusieurs logs.

4.1.2 Schéma relationnel

exemple de log modification :

Copie d'écran du modèle de log_modifications :

```python
# nrj_api/models/log_modifications.py

from django.db import models

class LogModifications(models.Model):
    log_id = models.AutoField(primary_key=True)
    utilisateur = models.ForeignKey('Utilisateurs', models.DO_NOTHING)
    fact_numero = models.CharField(max_length=50, blank=True, null=True)
    pdl_id = models.IntegerField(blank=True, null=True)
    fact_date = models.DateField(blank=True, null=True)
    date_action = models.DateTimeField(blank=True, null=True)
    action_effectuee = models.TextField()
    valeur_avant = models.TextField(blank=True, null=True)
    valeur_apres = models.TextField(blank=True, null=True)
    description = models.TextField(blank=True, null=True)
    delete = models.BooleanField(default=False)

    class Meta:
        managed = True
        db_table = 'log_modifications'

```

Ce modèle class est donc composé de plusieurs attributs chacun ayant une spécificité dans le bon fonctionnement de nôtre code. Par exemple, la valeur avant est un json avec une ancienne version de la facture) là ou valeur après est la version actuelle (au moment du log) de la facture. Ici la class Meta sert de règle lors de la connexion avec la table notamment en définissant le nom de la table grâce à “db_table” mais aussi si django va devoir gérer la table avec “managed”. Si django gère la table alors il va la créer et modifier les règles de ses colonnes.

4.2 Contraintes & validations

4.2.1 Contrainte d’unicité

- Clé primaire composite sur (fact_numero, pdl_id, fact_date, fact_conso_debut) empêche la création de doublons factures.

4.2.2 Contraintes CHECK exemple

- fact_montant > 0
- fact_consommation >= 0
- date_action <= NOW() pour les logs
- role IN ('Utilisateur','Auditeur','Admin') au niveau application

4.2.3 Validations côté Django

Dans le models/unmodelexemple.py :

4.3 Gestion des doublons

- Import/ ajout IHM: l'API backend pour l'import (POST /fait_facturation/) vérifie avec get_or_create() puis update() si déjà existant.
- API PATCH : /patch/:fact_numero/pdl/:pdl_id/date/:fact_date/ ne crée jamais de nouvelle entrée, mais met à jour les factures existant entre au travers de logs.
- Messages d'erreur : en cas de doublon tentant un nouveau POST, on renvoie HTTP 409 Conflict avec message explicite.

4.4 Performance & indexation

4.4.1 Indexes principaux

- Index unique implémente la primary key composite.
- Index non-unique sur (pdl_id, fact_date DESC) pour accélérer les recherches récentes par PDL.
- Index sur energie_id et tari_id pour les jointures.
- Index sur date_action dans log_modifications pour la pagination des logs.

4.4.2 Pagination

- Limit–offset par défaut :
  - Paramètres limit et offset gérés par LimitOffsetPagination de Django REST.
  - Maximum limit=1000 pour éviter les lectures massives.

4.4.3 Optimisation des requêtes

- Sélection des colonnes : utilisation de only() ou values() pour ne récupérer que les champs nécessaires.
- Préfetch des relations : select_related("pdl","utilisateur") pour éviter le N+1.

4.5 Backend : filtres et recherche

4.5.1 Filtrage simple

Dans le FacturationViewSet :

4.6 Retour critique & questions

- Points forts :
    - Structure normalisée, respect des bonnes pratiques Django.
    - Index adaptés pour les cas d'usage (recherche, pagination).
    - Gestion robuste des doublons via unique_together et endpoints PATCH.
- Axes d'amélioration :
    - Envisager une partition date sur fact_facturation pour alléger les scans sur plusieurs années.
    - Ajouter un audit trigger au niveau PG pour historiser toutes modifications.

4.5 Filtres & recherche

Pour rendre l'API de facturation flexible, nous utilisons :

- DjangoFilterBackend avec un FilterSet personnalisé (FaitFacturationFilter) qui gère :
    - ?fact_numero= (icontains)
    - ?pdl_id=, ?fact_date=, ?tari_id= (exacts)
    - ?pdl_numero= (recherche par numéro via sous-requête sur dim_pdl)
- SearchFilter pour rechercher dans fact_numero et pdl__adresse via ?search=.
- OrderingFilter pour trier sur fact_date, fact_montant, etc. avec ?ordering=.
- LimitOffsetPagination pour paginer via ?limit=/?offset=.

Endpoints spécialisés

En complément du simple listing (GET /api/nrj/fait_facturation/), nous exposons :

- GET .../latest_facturations/ : 5 factures les plus récemment modifiées.

- GET .../stats/ : { total_factures, montant_total }.
- GET .../dashboard/ : { thisMonthCount, thisMonthPaid, yearCount, yearPaid }.
- GET .../energie_types/ répartition factures par type d'énergie pour graphiques.

Mission 5 – Système de traçabilité

Pour garantir l'auditabilité des opérations et éviter toute confusion lors des modifications, nous avons mis en place un module de logs. L'objectif principal est que lorsqu'un auditeur modifie une facture, une trace précise soit conservée, évitant ainsi les ambiguïtés et assurant une transparence totale.

5.1 Modèle de données des logs

La table log_modifications conserve un historique complet :

Colonne	**Type**	**Description**
log_id	SERIAL PK	Identifiant unique.
fact_numero	VARCHAR	Clé de la facture.
pdl_id	INTEGER	Point de livraison.
fact_date	DATE	Date de la facture.
utilisateur_id	INTEGER	Auteur de l'action (par exemple, l'auditeur).
action_effectuee	VARCHAR	create / update / delete / rollback.
description	TEXT	Commentaire contextualisé (ex : "Montant modifié").
valeur_avant	JSONB	États des champs avant la modification.
valeur_apres	JSONB	États des champs après la modification.
date_action	TIMESTAMP	Horodatage précis de l'événement.
delete	BOOLEAN	Marqueur pour les suppressions (true si suppression de la ligne).

Ce schéma assure qu'à chaque action, quelle qu'elle soit, nous disposons d'une piste d'audit complète.

5.2 Création des logs

1. Suppression de facture

    - Lorsqu'un auditeur clique sur "Supprimer", le système vérifie d'abord qu'aucune suppression n'a déjà été enregistrée pour cette facture.
    - Si c'est la première suppression, un enregistrement est créé contenant:

        - L'identifiant de la facture (numéro, point de livraison, date).
        - L'identifiant de l'auditeur.
        - L'action effectuée ("suppression").

        - Une description libre (par exemple : "Facture supprimée suite à doublon détecté").
        - Une marqueur indiquant que c'est une suppression.
        - La date et l'heure précise de l'action.

2. Modification de facture

    - Pour chaque modification (changement de montant, de date ou d'un autre champ), l'interface front enregistre simultanément :
        - Les valeurs avant modification (par exemple montant initial).
        - Les valeurs après modification.
        - L'identifiant de l'auditeur.
        - Un libellé explicite (par exemple : "Montant passé de 1 200 € à 1 350 €").
        - La date et l'heure de la modification.

3. Création initiale

    - La création d'une nouvelle facture est aussi loguée, afin de disposer d'une trace complète de tout le cycle de vie de la donnée.
    - Le log indique que c'est une création, avec les valeurs initiales et le profil de l'auditeur ou de l'utilisateur.

5.3 Consultation des logs

- Vue chronologique

    - Dans la page de consultation d'une facture, une timeline liste toutes les actions dans l'ordre décroissant (de la plus récente à la plus ancienne).

- Chaque entrée précise :
  - La nature de l'action (create/update/delete/original).
  - Le nom et le rôle de l'utilisateur ayant réalisé l'action.
  - Le descriptif textuel saisi.
  - L'horodatage exact.

La nature originale signifie que ce n'est pas une facture ajouter depuis l'IHM mais depuis le service de donnée au travers de leur ETL (permettant donc de différencier d'où proviennent les factures).

- Ergonomie

  - Un code couleur et une icône distinguent visuellement les suppressions (rouge), les mises à jour (orange), les créations (vert) et les imports originaux (blanc).

  - Un bouton accessible en haut de la timeline permet de lancer un rollback de la dernière action si besoin.

5.4 Rollback partiel

Le rollback partiel vise à annuler uniquement la dernière modification sans perdre tout l'historique :

1. Bouton "Rollback"
   - Présent si au moins un log existe.
   - Intitulé clair : "Annuler la dernière action".

2. Processus côté serveur
   - On récupère le dernier log (hors rollback).
   - Si c'était une mise à jour, on restaure les champs à leurs valeurs avant.
   - Si c'était une suppression, on recrée la facture avec les valeurs historisées.
   - Un nouveau log "rollback" est alors enregistré, détaillant l'opération effectuée et les valeurs appliquées.

3. Actualisation

   - Après le rollback, la page se rafraîchit automatiquement, montrant la facture restaurée et mettant à jour la timeline pour refléter cette annulation.

5.5 Bénéfices et perspectives

- Gain de confiance interne : Les auditeurs savent que chaque action est tracée, ce qui réduit les litiges et les erreurs.
- Suivi simplifié : Les responsables peuvent rapidement vérifier qui a fait quoi et quand, sans avoir à fouiller dans plusieurs interfaces.
- Réversibilité maîtrisée : Grâce au rollback, on corrige les erreurs sans effacer l'historique complet, ce qui est crucial pour l'analyse suite à une suppression.

Axes d'amélioration possibles :

- Stockage des différences de valeurs (valeur_diff) plutôt que des objets complets, pour alléger la base de logs.
- Mise en place d'une interface centralisée de consultation de tous les logs par utilisateur ou période.

Mission 6 – Évolutions fonctionnelles et optimisation technique

Contexte initial

Dès la mise en pré-production de la première version de l'application, l'équipe métier a rapidement remonté des besoins d'ajout de fonctionnalités et d'amélioration de la navigation. Les retours concernaient par exemple d'une part la qualité des filtres et des tris disponibles, et d'autre part la performance de l'affichage des listes de factures. Mon objectif a été de répondre à ces demandes de façon concrète, tout en garantissant la stabilité de l'existant et la fluidité de l'expérience utilisateur.

Refactoring des appels API et modification de la clé de référence

Au cœur du projet, le défi majeur a porté sur la redéfinition de la clé de référence des facturations, qui s'appuyait initialement sur un simple numéro de facture. Très vite, les besoins métier se sont précisés : il ne suffisait plus de filtrer sur ce seul identifiant, mais il fallait garantir l'individualité de chaque enregistrement en combinant quatre éléments – le numéro de facture, l'identifiant du point de livraison (PDL), la date de facturation et la date de début de consommation. De mon côté, cela s'est réalisé par la mise en place d'une contrainte unique_together dans le modèle Django, assurant que chaque combinaison de ces quatre champs représente une facture distincte.
Cette évolution a bouleversé la logique de tous les endpoints existants. Là où les requêtes back-end se contentaient jusqu'alors de rechercher un enregistrement par numéro seul, il a fallu adapter la méthode get_queryset() au sein des ModelViewSet

pour incorporer la nouvelle combinaison de filtres. J'ai veillé à maintenir la compatibilité ascendante des URLs : les anciens appels REST, sans tous les paramètres, continuaient de fonctionner en renvoyant un jeu de résultats plus large, tandis que les nouveaux incluent systématiquement les quatre critères pour obtenir une facture précise.
Côté front-end, l'impact a été tout aussi important. Chaque service React qui appelait l'API devait maintenant transmettre les quatre paramètres dans l'URL — qu'il s'agisse de l'affichage d'une facture ou de son édition. Les routes définies dans React Router ont dû être redéfinies : là où l'on passait auparavant :fact_numero seul, les chemins sont devenus /invoice/view/:fact_numero/:pdl_id/:fact_date/:date_conso_debut. Cette transition a impliqué de renommer les props correspondantes, de modifier les composants pour récupérer les nouveaux paramètres depuis useParams(), et de tester manuellement chacune des combinaisons possibles pour s'assurer qu'aucune erreur n'apparaissait.
En définitive, la mise en place de la contrainte unique_together a renforcé l'intégrité des données et clarifié la relation univoque entre les factures et leur contexte, mais elle a également nécessité une refonte en profondeur de la chaîne de traitement, du modèle Django à l'IHM React, pour que chaque appel et chaque route reflètent fidèlement ce nouveau mode de référence.

Enrichissement de l'IHM et adaptation des routes

Au-delà du simple filtrage, la grille de résultats devait désormais afficher le site et la collectivité associée à chaque enregistrement. J'ai étendu le modèle Django pour intégrer ces champs, puis j'ai mis en place une jointure avec la table PDL, en utilisant select_related pour limiter le nombre de requêtes SQL. Le serializer a été modifié automatiquement pour exposer ces nouvelles colonnes dans le JSON.
Dans l'interface React, la grille MUI s'est enrichie de deux nouvelles colonnes, positionnées à droite de la date de facture. Pour permettre un tri multi-critères (date, site, montant), j'ai réécrit la logique de tri front-end de façon à gérer l'ordre de priorité des colonnes affichées. Cette refonte a également impliqué la mise à jour des hooks d'état, afin que les tris interactifs informent correctement le back-end des nouveaux paramètres d'ordering.

Extension des critères de recherche

Au cours des sessions de recettage, les utilisateurs ont exprimé le souhait de filtrer la liste des factures selon des intervalles de montant et par type d'énergie. J'ai complété le FilterSet avec un filtre de plage pour le montant et un filtre de sélection multiple pour le type d'énergie, alimenté dynamiquement depuis la table de

référence dim_energie. Dans l'IHM, j'ai remplacé l'ancien champ de recherche unique par un panneau de filtres rétractable, qui préserve l'espace écran tout en offrant un contrôle granulaire.
Chaque critère supplémentaire a conduit à des tests de bout en bout : s'assurer que la recherche par montant ne renvoie pas de résultats hors plage, et que la combinaison de filtres conserve la cohérence du jeu de données.

Optimisation des performances et fiabilité

Malgré ces enrichissements, plusieurs tickets signalent des lenteurs récurrentes dans le chargement de la liste de factures, surtout lors de la navigation rapide entre pages. Pour y remédier, j'ai d'abord centralisé tous les appels API dans un hook personnalisé useFactures, supprimant ainsi les appels redondants qui survenaient lorsque plusieurs composants déclenchent simultanément la même requête.
Ensuite, j'ai implémenté une mise en cache locale des données statiques (liste des points de livraison et des types d'énergie) via la Context API de React. Ces listes, auparavant rechargées à chaque navigation, sont désormais chargées une seule fois et partagées entre tous les composants, ce qui supprime plusieurs requêtes réseau inutiles. Enfin, j'ai limité la taille des réponses JSON en restreignant les champs renvoyés par l'API aux seules colonnes visibles à l'écran. Ces optimisations, validées qualitativement par l'équipe métier, ont considérablement fluidifié la navigation et réduit les risques de blocage de l'interface.

Coordination, recette et déploiement

Tout au long de cette mission, j'ai assuré une collaboration avec les référents métier via Teams, planifiant des démonstrations hebdomadaires pour présenter les nouvelles fonctionnalités. Les tickets Redmine ont servi de référence pour prioriser les corrections et priorités. Pour chaque version livrée en staging, j'ai préparé les migrations Django nécessaires à la base de données, sollicité les droits d'exécution via GLPI et validé en recette la bonne application des évolutions. Les retours des sessions UAT ont été intégrés dans la version finale, déployée en production sans incident majeur.
Grâce à une approche itérative, rigoureuse et orientée utilisateur, cette mission a permis de transformer les premiers retours de recettage en une IHM riche, performante et stable, répondant pleinement aux besoins métiers sans compromettre la qualité technique.

II. Valorisation des compétences

Partie 2 - Projet Scolaire : WildLens

Partie 1 — Compréhension et cadrage des besoins métiers

1.1 Contexte global des trois projets IA & Data

Au cours de ma formation, j'ai travaillé sur trois parties successives autour du développement d'une solution IA de reconnaissance d'empreintes animales, correspondant aux rapports MSPR1 (modélisation des données et ETL), MSPR2 (mise en production et CI/CD) et MSPR3 (modélisation avancée, entraînement et déploiement). Chacune de ces phases s'inscrivait dans un cadre précis :

MSPR1 (18 déc. 2024 – 19 mars 2025) : définir et mettre en place le modèle de données exploitable par le modèle IA, automatiser l'ETL depuis la collecte de sources hétérogènes (iNaturalist, NatureTracking, OpenAnimalTracks, Google Images) jusqu'à la transformation de ces données en parquet puis enregistrement dans la base PostgreSQL.

MSPR2 (20 mars 2025 – 4 juin 2025) : transformer le prototype en service déployable, concevoir l'API backend, dockeriser la chaîne IA (modèle, base de données, front-end) et intégrer un pipeline CI/CD complet (tests unitaires, sécurité, déploiement continu).

MSPR3 (5 juin 2025 – 4 juil. 2025) : implémenter et comparer plusieurs architectures de classification (CNN, MobileNetV2, ViT), affiner les hyperparamètres, documenter l'Active Learning et déployer le modèle via une API FastAPI.

Ces trois étapes obéissent au même fil rouge : répondre à un besoin métier clair — identifier des empreintes animales — tout en structurant rigoureusement l'ensemble des données, en garantissant la portabilité et l'automatisation, et en assurant la traçabilité des résultats.

1.2 Recueil et formalisation du besoin (BC04)

Identification des acteurs et des enjeux
À partir des échanges avec nos professeurs, j'ai formalisé un cahier des charges dans lequel figurent :

- Les sources de données (open data, scrapping, fichiers fournis par l'école) ;

- Les contraintes techniques (volumes à ingérer, fréquence de mise à jour, normes RGPD pour l'anonymisation) ;
- Les livrables à chaque jalon (schéma UML, API REST, Docker Compose, notebook d'entraînement, documentation Swagger, rapports de projet).

Cette étape d'organisation a permis de fixer des objectifs (ex. : automatiser l'ETL pour 3 000 images en moins de 30 mn ; atteindre 80 % de F1-score macro avant la phase de production).

1.3 Analyse des sources et contraintes techniques (*BC01*)

Définition des sources et outils

- Images d'empreintes : OpenAnimalTracks (2 335 images), iNaturalist et NatureTracking via Selenium pour naviguer dans les galeries, Google Images avec respect du robots.txt.

- Métadonnées espèces : fichier CSV initial + deux nouveaux fichiers récupérés en ligne regroupant plus de 200 mammifères, enrichi par API Mistral AI pour compléter description, habitat, fun fact …

- Outils :
  - Extraction : Selenium/WebDriver, Requests/BeautifulSoup, validation des formats et détection de corruption via OpenCV.
  - Transformation : PySpark pour manipuler de larges volumes, Pandas pour affinage.
  - Stockage : CSV transformé en Apache Parquet pour l'optimisation et la manipulation puis PostgreSQL conteneurisé pour l'échange avec le back.

J'ai mis en place des scripts pour chaque source, orchestrés par un pipeline Python qui :

1. Vérifie les permissions robots.txt avant tout scrapping.
2. Téléchargez et filtrez les images (JPEG/PNG < 10 Mo), relancez les requêtes en cas d'échec (3 essais).
3. Insère automatiquement les fichiers validés en format Parquet, en parallèle d'un journal d'activité.

Ces routines assurent la reproductibilité et la sécurisation des données dès leur collecte.

1.4 Qualification et tri des données (*BC01*)

Avant l'ingestion finale, j'ai procédé à une étape de profiling :

- Détection et suppression des doublons (drop_duplicates/mixup checksums).
- Nettoyage des métadonnées (suppression des accents mal encodés, uniformisation des libellés, normalisation des unités métriques).
- Mise en place de fonctions de data quality (regexp_replace pour les champs texte, dropna pour les valeurs critiques).
- Visualisations exploratoires (histogrammes de distribution d'images par espèce, heatmaps d'échantillonnage) pour identifier les classes sous-représentées et planifier l'augmentation.

1.5 Retour critique sur le cadrage (BC05)

- Points forts :
  - Un cahier des charges réaliste et validé dès MSPR1 a limité les dérives fonctionnelles.
  - L'automatisation de l'ETL a maintenu la qualité des données tout au long du projet.

- Axes d'amélioration :
  - Intégrer en amont un schéma de la donnée (catalogue, glossaire) pour faciliter la collaboration .
  - Formaliser davantage les cas d'usage métiers (ex. : alertes sur classes rares) pour affiner les métriques d'évaluation dès la phase de cadrage.

Partie 2 — Conception, entraînement et évaluation de la solution IA

La deuxième phase de notre projet a été passionnante : transformer notre jeu de données préparé en un modèle de classification capable de reconnaître automatiquement des empreintes animales. L'objectif était de combiner rigueur scientifique et exigences opérationnelles pour proposer un algorithme performant face aux variations d'images, et suffisamment léger pour être déployé dans une API industrielle.

2.1 Du corpus brut à l'entrée réseau (*BC02)*

La première étape a été de standardiser les images brutes, dont les résolutions et qualités varient énormément, allant du cliché flou pris avec un smartphone aux scanners haute définition. Plutôt que d'imposer un cadre rigide, j'ai conçu une chaîne TensorFlow pour lire et convertir les images en format RG, puis les redimensionner à 384×384 pixels. Ce choix de résolution, inspiré par le pré-entraînement du ViT (Vision Transformer) sur des patchs de 16×16, permettait de conserver suffisamment de détails pour distinguer des motifs subtils tout en limitant la consommation de mémoire GPU. La normalisation des pixels dans l'intervalle [–1, 1] a assuré une convergence plus rapide et plus stable du modèle, évitant ainsi les saturations de gradient.

Pour maximiser la capacité de généralisation, j'ai intégré un module d'augmentation dans la pipeline data_pipeline.py. Les rotations aléatoires (±15°), les zooms légers, les inversions horizontales et les variations de luminosité simulaient des prises de vue en conditions variées. Des techniques plus sophistiquées comme Mixup et CutMix, activées à la volée, composaient des images synthétiques en combinant deux empreintes au hasard, renforçant ainsi la diversité sans stocker d'images supplémentaires. Cette approche, optimisée pour le GPU grâce à .cache() et .prefetch() sur les tf.data.Dataset, a réduit de près de moitié le temps consacré à la préparation des données par lot, sans pénaliser l'entraînement.

2.2 Choix des architectures et justification (*BC02*)

Au démarrage du projet, j'ai listé trois familles d'architectures à comparer : un CNN de base entraîné de zéro (baseline), deux modèles de transfer learning (MobileNetV2 et ResNet50), et un ViT large pré-entraîné sur ImageNet21k. L'objectif n'était pas seulement d'obtenir la meilleure performance brute, mais aussi de comprendre leurs comportements en termes de rapidité d'entraînement, de robustesse aux classes rares et de capacité à interpréter les décisions.

Le CNN baseline, composé de cinq couches de convolution suivies de pooling et d'une tête dense, a servi de référence. Il a permis de valider le bon fonctionnement de la chaîne d'entraînement et d'isoler l'apport des modèles pré-entraînés. En pratique, ses performances plafonnaient autour de 65 % d'accuracy pour 58 % de F1-score, révélant une sensibilité excessive aux artefacts du fond et aux variations d'éclairage.

Les modèles MobileNetV2 et ResNet50, gelés en tête dans un premier temps, ont été fine-tunés en deux phases. D'abord, seules les couches de classification étaient entraînées, avec un learning rate de 1e-3 pendant dix époques, puis, après dégel progressif, l'ensemble du backbone l'a été à un taux plus faible (1e-4). Cette stratégie a permis d'atteindre respectivement 81 % et 83 % d'accuracy, tout en conservant un entraînement relativement rapide (moins de 1 heure sur GPU RTX

4070). Cependant, ces réseaux pré-entraînés restaient quelque peu biaisés par les caractéristiques visuelles de l'ImageNet d'origine, peinant à dissocier certaines espèces dont les empreintes se ressemblent.
Le modèle Vision Transformer (google/vit-large-patch16-384-in21k), quant à lui, nécessite une préparation un peu différente : chaque image était découpée en patches de 16×16, linéarisée et dotée d'embeddings de position. En exploitant l'API TFAutoModel de Hugging Face, j'ai ajouté une couche dense de 128 neurones activée ReLU au sommet de la sortie CLS, suivie d'un dropout à 0,3 et d'une dernière couche softmax. L'optimiseur AdamW, couplé à un schedule CosineDecay, a assuré un équilibre entre exploration (warmup) et fine-tuning. Au terme de 25 époques, le ViT dépassait largement les autres architectures avec 89 % d'accuracy et un F1-score macro de 86,5 %, tout en montrant une résistance marquée aux déséquilibres de classes et aux images partiellement corrompues.

2.3 Stratégie d'entraînement et paramètres clés (*BC02*)

L'entraînement s'est déroulé en deux grandes phases. D'abord, l'"initial training" de cinq époques, où le backbone des grands modèles restait gelé, a stabilisé les poids de la tête de classification. Les callbacks ModelCheckpoint, EarlyStopping (patience = 3), et ReduceLROnPlateau (monitor = val_loss, facteur = 0,5) minimisaient le risque de surapprentissage. Chaque lot était de taille 32, ajusté pour optimiser le débit GPU et éviter l'out-of-memory.
La phase de fine-tuning a libéré progressivement chaque couche du backbone, avec un learning rate plus faible de 1e-5 pour les premières couches et 5e-5 pour les dernières, selon un plan de gel/dégel linéaire. Cette méthode, combinée à la prise en compte des poids de classe inverses dans la fonction de perte (class_weight calculés depuis la distribution réelle des images), permettait de mieux appréhender les classes rares.
Les métriques suivies en temps réel comprenaient l'accuracy globale, le F1-score macro via une fonction custom implémentée dans la boucle tf.keras.metrics, et la balanced accuracy. La matrice de confusion était générée à chaque évaluation sur l'ensemble de validation, facilitant l'identification des confusions fréquentes entre espèces proches.

2.4 Évaluation détaillée et interprétabilité (*BC02*)

L'analyse expérimentale a clairement positionné le ViT en tête : non seulement il atteignait un F1-score supérieur de 7 points par rapport à MobileNetV2, mais il conservait une plus grande stabilité sur les classes les moins représentées. Les CNN et MobileNet souffraient de biais liés à la texture du sol : boue ou sable, qui étaient occasionnellement confondus avec certaines empreintes.

Pour mieux comprendre ces phénomènes, j'ai implémenté une version adaptée de Grad-CAM, en extrayant non plus les activations de convolution (absentes dans le ViT) mais les cartes d'attention des dernières couches de transformation. Les heatmaps superposées aux images d'entrée ont révélé que le CNN se focalise souvent sur des artefacts périphériques (ombres ou débris), tandis que le ViT accordait un poids plus global à la forme de l'empreinte, même si elle était partiellement occultée. Dans les cas d'erreur, les zones mises en valeur restaient logiques — le modèle se trompait sur des empreintes géométriquement semblables —, attestant de sa cohérence interne.

2.5 Intégration dans une API et perspectives de production (*BC03*)

Dès l'atteinte des performances visées (F1-score > 85 %), la phase suivante a été la mise à disposition du modèle sous forme de service. J'ai développé une API FastAPI qui, à chaque appel POST /predict, réceptionnait une image encodée en multipart/form-data, appliquait la même pré-pipeline (redimensionnement, normalisation), puis renvoyait un JSON structuré avec la classe prédite, le score de confiance et, en option, la heatmap Grad-CAM renvoyée sous forme de base64. L'API, conteneurisée via Docker, se chargeait dès le démarrage de TFViTModel.from_pretrained, réduisant le temps de réponse à 200–300 ms par requête en mode production.
Afin d'assurer la stabilité, j'ai également préparé un plan de tests automatisés via PyTest :

- Tests unitaires pour chaque route et chaque fonction de pré-traitement ;
- Tests de performance (via Locust) pour simuler une centaine de requêtes concurrentes ;
- Tests de non-régression comparant le F1-score du modèle en production à un baseline stocké dans un .npz.

Enfin, pour favoriser l'amélioration continue, j'ai documenté l'API avec Swagger UI intégré à FastAPI, et exposé les endpoints de monitoring (via /health et /metrics) pour collecter des indicateurs Prometheus. Cette approche permet de compléter le déploiement effectif du modèle IA au sein d'une solution end-to-end, conforme aux meilleures pratiques DevOps et Data Science.

Partie 3 — Gestion des activités et tâches du projet IA

1. Méthodologie de pilotage (Kanban & Gantt) (*BC04*)

Dès le lancement de chaque phase, l’équipe a adopté une organisation agile, combinant un tableau Kanban sous Trello pour le suivi quotidien et un diagramme de Gantt pour la vision à moyen terme.

- Trello Kanban : chaque fonctionnalité (ETL, Fine-tuning, CI/CD, UI, etc.) était découpée en cartes « À faire », « En cours », « Bloqué » ou « Terminé ». Cette granularité a permis de détecter et lever rapidement les blocages (ex. erreurs CUDA, permissions Android), et de répartir les tâches entre les membres selon leur expertise .

- Diagramme de Gantt : planifié sur Excel puis intégré dans les rapports MSPR, il fixait les jalons clés (collecte de données, entraînement initial, tests de non-régression, déploiement). Ce repère temporel a permis d'ajuster la charge de travail et de coordonner les dépendances (ex. fin du prétraitement avant entraînement, déploiement après tests CI) .

Chaque itération était conclue par un point d'équipe hebdomadaire (scrum-like) : on validait les tâches « Terminées », priorisait les nouveaux tickets et mettait à jour le Gantt en fonction de l'avancement réel.

2. Répartition des responsabilités et montée en compétence (*BC04*)

Au sein de l'unité Projet & Développement, chaque membre avait un périmètre précis :

- Collecte et prétraitement (ETL) : implémentation de scripts PySpark et Pandas pour extraire, nettoyer et structurer 3 275 images d'empreintes, avec gestion des fuites de données et équilibrage de classes .

- Modélisation et entraînement : comparaison CNN, transfer learning et ViT, réglage des hyperparamètres avec callbacks personnalisés (EarlyStopping, ReduceLROnPlateau, GradualUnfreeze) .

- Déploiement CI/CD : création de pipelines GitLab automatisant lint, tests unitaires, build et déploiement sur Railway/GitLab, assurant packaging Docker et contrôles de santé post-déploiement .

- Front-end & API : développement React/Flutter et FastAPI, structuration des routes, permissions RBAC, traçabilité via logs, et intégration du SSO Métropole via JWT et refresh tokens .

Cette répartition a favorisé la montée en compétence transverse : un développeur back-end a pu mettre la main sur la configuration GitLab CI, tandis que les spécialistes IA ont assisté à l'optimisation des requêtes Django ORM.

3. Gestion des imprévus (*BC04*)

Plusieurs aléas ont été gérés via la méthodologie agile :

- Blocages techniques (cuda_malloc_async, incohérences d'API) résolus en daily calls et recherche de documentation (Swagger, logs Grafana) .

- Retards de dépendances (participant indisponible, accès base DB) comblés par re-priorisation des tâches dans Trello et ajustement du planning Gantt.

- Basculement en environnement de secours : mise en place d'un container PostgreSQL de test pour développer localement sans impacter la base prod .

Partie 4 – Communication et veille technologique

1. Communication interne et documentation (*BC05*)

La cohésion d'équipe et le transfert de connaissances se sont appuyés sur :

- Wikis et ReadMe dans chaque dépôt GitLab, détaillant l'installation Docker, les scripts ETL et les endpoints API .
- Compte-rendus systématiques après chaque réunion hebdomadaire (analyse besoins, revue de code, sprint review), stockés sur Google doc & Discord pour garantir la traçabilité des décisions métier et techniques .
- Démonstrations et revue avec nos professeurs pendant les heures de cours attitrés à la MSPR pour récupérer des avis et conseils.

2. Veille technologique (*BC05*)

Pour rester à jour sur les bonnes pratiques IA et DevOps :

- Surveillance des releases des frameworks (TensorFlow, HuggingFace Transformers, Django REST) via GitHub Watch & X .
- Abonnement aux newsletters (e.g. Towards Data Science, PyData, GitLab CI Blog) et suivis des comptes de release sur les réseaux sociaux (LinkedIn & X), partage des informations dans un serveur discord dédié au projet.
- Tests de nouvelles bibliothèques (Mistral AI pour la complétion des métadonnées, Grad-CAM avancé pour ViT) en POC isolé, avec compte-rendu et note dans la documentation interne .

3. Amélioration continue (*BC05*)

Cette veille active a permis :

- D'optimiser les pipelines GPU (activation de cuda_malloc_async, mixed_float16) réduisant de 40 % la charge mémoire .
- D'anticiper la mise à jour de Django pour la v4 et préparer le passage à FastAPI v.0.100.
- D'intégrer des outils de monitoring (Prometheus + Grafana) pour visualiser en temps réel la latence API et les performances GPU en production .

III. Conclusion et Perspectives

Perspectives pour la Métropole de Montpellier :

La Métropole de Montpellier se distingue par son engagement dans la modernisation et l'innovation, notamment à travers des projets comme l'Interface Homme-Machine (IHM) de gestion des factures. Cette initiative transforme profondément les pratiques des équipes Énergie de la ville en centralisant la saisie et la modification des factures directement en ligne. Elle élimine les heures fastidieuses passées à ressaisir des données dans Excel, réduit les erreurs humaines et supprime les incompatibilités liées aux formats propres à chaque fournisseur. L'intégration avec Business Objects offre désormais aux agents un outil de restitution professionnelle, leur permettant de visualiser et d'analyser leurs factures via des tableaux de bord avancés. Cette modernisation constitue un véritable gain de temps et ouvre la voie à une homogénéisation du traitement des factures dans d'autres services tels que l'eau, les déchets ou la voirie.
Sur le plan métropolitain, il serait bénéfique de généraliser l'IHM de facturation à l'ensemble des services métier. Cela implique de proposer un front unifié et des procédures communes, accompagné d'un plan de formation et de supports utilisateurs pour faciliter l'adoption. Le marché actuel, porté par la transformation numérique, offre des opportunités significatives pour renforcer les services publics et optimiser les processus internes.

Axes d'amélioration :

Du côté du service développement, l'expérience acquise montre qu'il serait judicieux de mettre en place un squelette référenciel incluant, par exemple, un module de login générique, des Dockerfiles préconfigurés et des pipelines CI/CD standards. La création d'une bibliothèque de composants réutilisables pour l'interface utilisateur,

l’automatisation des scripts et les tests unitaires permettrait d’accélérer la mise en œuvre des projets et d’assurer une cohérence technique entre les différentes applications métiers.
Le service Énergie de la Métropole de Montpellier joue un rôle crucial dans la gestion des ressources et l'optimisation des coûts. Avec l'introduction de l'IHM, ce service est en voie de modernisation, ce qui pourrait servir de modèle pour d'autres services. À l'avenir, ce service pourrait se développer davantage, en intégrant des outils supplémentaires pour une gestion plus unifiée et efficace.

Apports professionnels et personnels :

Travailler au sein de la Métropole de Montpellier a offert l’opportunité de consolider des compétences techniques et méthodologiques essentielles. La diversité des projets, allant de la conception de l'Interface Homme-Machine (IHM) à la mise en place des pipelines CI/CD, a permis d’approfondir la maîtrise de Docker, GitLab, et des frameworks front-end et back-end, tout en renforçant la rigueur dans le respect des bonnes pratiques DevOps. En intervenant directement sur le cycle complet de vie des applications, une vision globale des enjeux d’exploitation et de maintenance a été acquise, ce qui aide aujourd’hui à anticiper les besoins opérationnels et à proposer des solutions pérennes.
Sur le plan organisationnel, l’expérience au sein d’une équipe pluridisciplinaire a sensibilisé à l’importance de la communication et de la collaboration. Cela a permis d'apprendre à vulgariser des concepts techniques pour des utilisateurs métier, à structurer des retours d’expérience, et à coordonner les phases de tests, de déploiement et de formation. Cette posture a non seulement permis de renforcer la cohésion d’équipe, mais aussi d’affiner la capacité à piloter un projet de bout en bout, du cadrage fonctionnel à l’accompagnement au changement.
Enfin, le contexte de la Métropole a sensibilisé aux enjeux de transformation numérique à l’échelle territoriale, notamment en termes de performance énergétique et de service public. Contribuer à des outils qui améliorent concrètement le quotidien des agents et des citoyens a apporté une réelle motivation et un sentiment d’utilité sociale. Ces expériences ont par ailleurs nourri une curiosité pour les innovations à venir en intelligence artificielle et en science des données, et ont confirmé dans l'ambition de participer à des projets à forte valeur ajoutée pour les collectivités.

Annexes

Code de connexion à l'api3m pour la récupération / l'authentification d'un agent.

```python
class APIAuthenticationBackend(BaseBackend):
    def authenticate(self, request, username=None, password=None, **kwargs):
        print(f"Authenticating with identifiant/email: {username}")
        api3m_url = API3M_URL
        parsed_url = urlparse(api3m_url)
        if not parsed_url.scheme:
            api3m_url = f"https://{api3m_url}"
        api_url = f"{api3m_url}/api/authenticate-gia/"
        headers = {'Authorization': f'Token {API3M_TOKEN}'}
        payload = {
            "user": username,
            "password": password
        }

        try:
            response = requests.post(api_url, json=payload, headers=headers, verify=False)
            response.raise_for_status()
            print(f"Response status code: {response.status_code}")
            print(f"Response content: {response.json()}")

            if response.json().get("success"):
                user_info = response.json().get("user_info")
                uid = user_info["uid"]
                email = user_info["mail"].lower()

                print(f"✅ Authentifié via API3M : uid={uid}, email={email}")

                # Vérification dans la base PostgreSQL : uid + email doivent correspondre
                try:
                    utilisateur = Utilisateurs.objects.get(uid=uid, email=email)
                    print("✅ Utilisateur trouvé localement avec uid et email correspondants")
                except Utilisateurs.DoesNotExist:
                    raise AuthenticationFailed(
                        "Utilisateur non autorisé : uid et email ne correspondent pas localement."
                    )
```

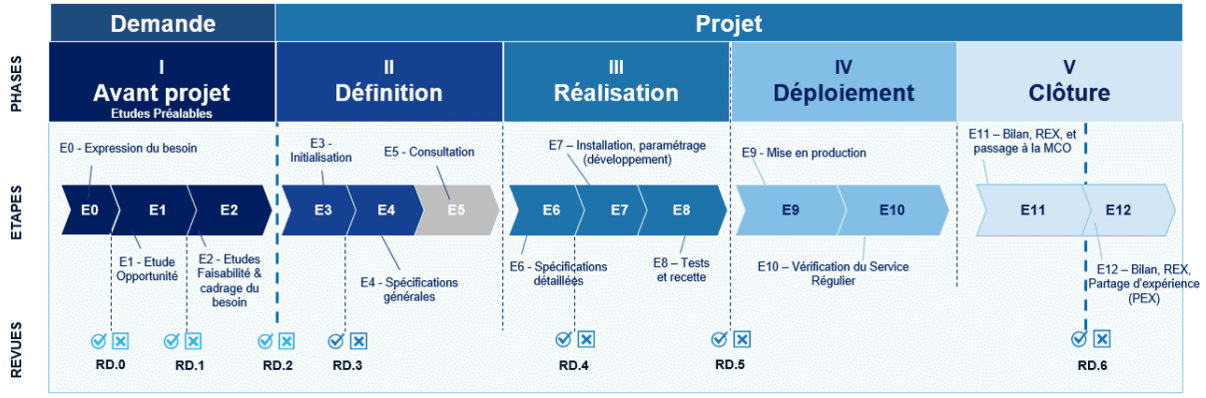



Diagramme de l'organisation d'un projet à la métropole que l'on m'a présenté lors du début de mon stage.